CSC241 / BAI / LECTURE 15

Method Overriding and Overloading

Same method name, different reasons

Dr Muhammad Shahid Bhatti
COMSATS University Islamabad, Lahore Campus

From Lecture 14 to Lecture 15

Last lecture

Constructor chaining initializes superclass state first.

protected members give subclasses controlled access.

final can prevent method overriding.

Today

A subclass can replace an inherited method implementation.

A class can also define several methods with one name and different parameter lists.

Lecture 15 scope from the CDF

Method overriding

A subclass provides a matching method implementation.

Overloading versus overriding

Compare the purpose, method signature, and context for each.

Polymorphism and dynamic binding appear later in the CDF.

Learning outcomes

By the end, you can

Explain overriding using a superclass and subclass.

Use @Override to document and check intent.

Call the superclass version with super.method().

You will also be able to

Distinguish overriding from overloading.

Recognize invalid overloads and overrides.

Predict which overload a call selects from its arguments.

A Person already has an introduction

Person

introduce()

Returns a general introduction.

Student

A Student is a Person.

Should it use the general introduction or describe itself as a student?

We will answer by overriding one inherited method.

A method signature identifies its name and parameters

```
introduce(String greeting)
```

Included

Method name

Parameter types and their order

Not part of the signature

Return type

Parameter variable names

Access modifier

Overloading gives one class several method forms

Same method name, different parameter lists

```
introduce()
```

```
introduce(String greeting)
```

```
introduce(String greeting, String program)
```

The compiler uses the argument count and types to find a matching overload.

Overloaded methods accept different arguments

```
// Person.java
public class Person {
    public String introduce() {
        return "I am a person.";
    }

    public String introduce(String greeting) {
        return greeting + ", I am a person.";
    }
}
```

KEY IDEA

Both methods belong to Person.

Their parameter lists differ.

The return type is the same, but that is not what makes them overloads.

The argument list distinguishes overloaded calls

Call	Matching method
`person.introduce()`	introduce()
`person.introduce("Hello")`	introduce(String)
`person.introduce("Hello", "AI")`	introduce(String, String)

Return type alone cannot create an overload

```
int calculate(int value)

double calculate(int value)
```

These declarations have the same name and parameter types.

Java cannot choose between them from the call calculate(5).

Overriding replaces an inherited implementation

Superclass

Person defines introduce().

It supplies general behavior.

Subclass

Student extends Person.

Student declares introduce() with the same parameter list and supplies student-specific behavior.

The subclass method matches the inherited signature

```
// Person.java
public class Person {
    public String introduce() {
        return "I am a person.";
    }
}

// Student.java
public class Student extends Person {
    @Override
    public String introduce() {
        return "I am a student.";
    }
}
```

KEY IDEA

Same method name.

Same parameter list.

Student gives the inherited operation a new implementation.

@Override asks the compiler to check your intent

With @Override

The compiler checks that the method actually overrides an inherited method.

Without @Override

A typo or parameter mismatch may create a new method instead of overriding the intended one.

Prefer @Override whenever you intend to override.

The parameter list must match

Superclass method

```
public String introduce() { ... }
```

Subclass method

```
@Override
public String introduce(String greeting) { ... }
```

This is a different parameter list. It does not override introduce().

With @Override, the compiler flags the mismatch.

super.method() can reuse the superclass behavior

```
// Student.java
public class Student extends Person {
    @Override
    public String introduce() {
        return super.introduce()
               + " I am also a student.";
    }
}
```

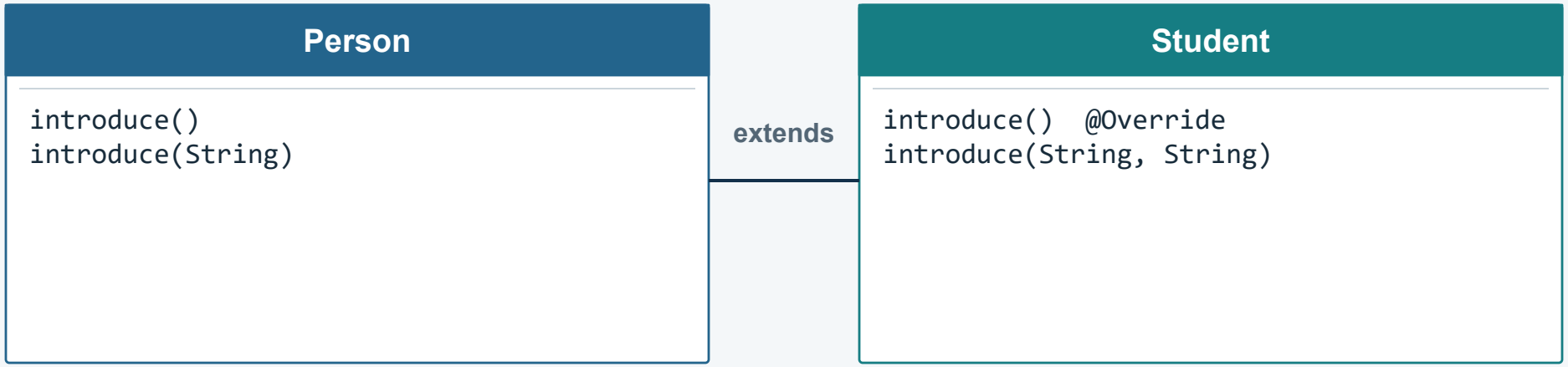
KEY IDEA

super.introduce() calls Person’s implementation.

Student then adds its own information.

Output: I am a person. I am also a student.

A Student object can use both inherited and declared overloads



Student overrides one form and adds another overload.

Overloading and overriding solve different problems

Overloading	Overriding
Same class (commonly)	Subclass and superclass
Same method name	Same method name
Different parameter lists	Same parameter list
Compiler matches the call arguments	Subclass replaces inherited behavior

Overriding keeps a compatible method contract

1. Use the same method name and parameter types.
2. Use the same return type, or a compatible subtype return type.
3. Keep the same access level or make access more open.
4. Do not override a final method.

An override cannot reduce visibility

```
Person: public String introduce()
```

```
Student: protected String introduce()
```

This override is invalid because callers would lose access.

final from Lecture 14 still applies

final method

The superclass fixes this method’s implementation.

A subclass cannot override it.

Example

```
public final String getUniversity() {
    return "CUI";
}
```

A matching method in Student causes a compile-time error.

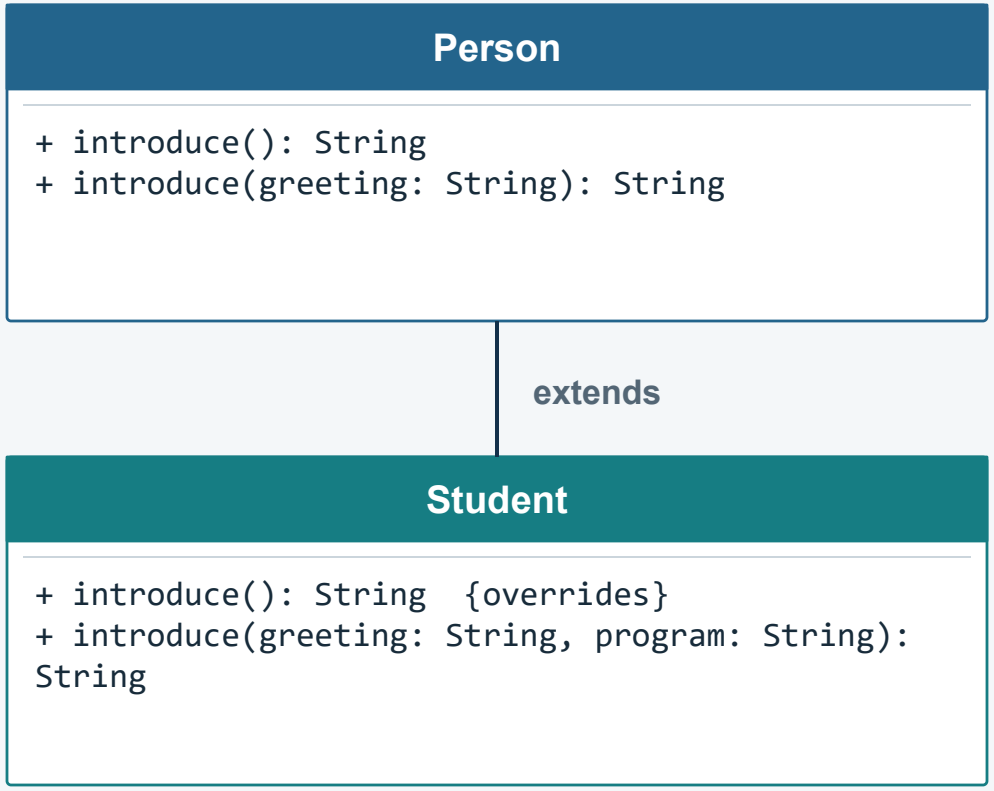
Some similar-looking methods are not overrides

Constructor	Constructors are not inherited and cannot be overridden.
private method	A private method is not inherited by the subclass.
static method	A matching static method hides the superclass method.
final method	A final method cannot be overridden.

Common mistakes and their fixes

Wrong parameter list	Add the exact parameter list to override.
Return type only changed	Use a different parameter list for an overload.
Visibility reduced	Keep access the same or widen it.
Misspelled method name	Add @Override so the compiler catches it.
Overriding a final method	Remove the override or change the superclass design.

Read overriding and overloading in UML



A UML class box lists the methods; the inheritance line identifies the override relationship.

Demo calls show the two ideas separately

```
Student student = new Student();

System.out.println(student.introduce());
System.out.println(student.introduce("Hello"));
System.out.println(
    student.introduce("Hello", "Artificial Intelligence")
);
```

KEY IDEA

Call 1 uses the overridden no-argument method.

Call 2 uses the inherited introduce(String) overload.

Call 3 uses Student’s new overload.

Predict each call before running Demo

```
student.introduce()
```

```
student.introduce("Hello")
```

```
student.introduce("Hello", "AI")
```

Expected output

```
I am a student.
Hello, I am a person.
Hello, I study AI.
```

One call is overridden. The other calls select different overloads.

Guided practice: add an override and an overload

Starting point

Person has getDescription() returning a general message.

Student extends Person.

Your task

Override getDescription() for a student.

Add getDescription(String course) as an overload.

Use @Override and predict both outputs.

Next in the CDF: Object and toString()

Lecture 15

Method overriding
Overloading comparison

Lecture 16

Object class
toString()

Overriding a method from Object will be our bridge to toString().

Overriding and overloading

Overriding changes inherited behavior.

Overloading provides another parameter list.

Name + parameters tell the compiler which method form you declared and called.